

Webshop Playwright Test Suite

Automated end-to-end and API-level tests for the Webshop React + Express application.

The suite uses **pytest** with the **pytest-playwright** plugin and is designed to surface all 23 known bugs documented in the project's [BUGS.md](#).

11 files · ~2,500 lines of code · **33 test cases**

Suite Overview

File	Purpose
requirements.txt	Python dependencies: pytest, pytest-playwright, pytest-html, playwright
pytest.ini	Full config: screenshots on, video retained on failure, HTML report auto-generated, 28 custom markers
conftest.py	Shared fixtures (base_url , authenticated_page , authenticated_page_user1), helpers (add_product_to_cart , complete_shipping_step , complete_payment_step), screenshot-on-failure hook
test_auth.py	TC_AUTH_001–009 — Registration, login, logout, password policy, cart on logout
test_products.py	TC_PROD_001–009 — Images, price filters, sorting, out-of-stock badge, mobile button
test_cart.py	TC_CART_001–007 — Add to cart, quantity limit, tax rate, discount codes, total update, logout
test_checkout.py	TC_CHECKOUT_001–005 — Auth gate, CVV, expiry, confirmation message, order history
test_database.py	TC_DB_001–003 — Negative quantity, cross-user orders, password hashing (API-level)
run_tests.sh	Linux / macOS runner script
run_tests.bat	Windows runner script
README.md	This file

Expected results on the buggy app

Result	Count	Meaning
FAIL	25	Bug confirmed — assertion message includes Bug ID
PASS	7	Correct baseline behaviour
XFAIL	1	TC_DB_003 — MD5 hashing, cannot verify via API alone

Prerequisites

Tool	Minimum version
Python	3.9+
pip	latest
Node / Docker	only needed to run the app itself

Setup

```
# 1. (Recommended) Create and activate a virtual environment
python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS / Linux
source .venv/bin/activate

# 2. Install Python dependencies
pip install -r requirements.txt

# 3. Install the Chromium browser used by Playwright
playwright install chromium
```

Target Environments

Name	URL	How to target
local	http://localhost	Default — local Docker compose
dev	http://localhost:5173	Vite dev server (<code>npm run dev</code>)
prod	https://webshop.xpert-hub.com	Deployed production server

Running the Tests

Windows — quick commands

```
:: Local Docker (default)
run_tests.bat

:: Local Docker (explicit)
run_tests.bat --local

:: Vite dev server
```

```
run_tests.bat --dev

:: Production
run_tests.bat --prod

:: Extra pytest args still work
run_tests.bat --prod -m smoke
run_tests.bat --prod test_auth.py -v
```

Linux / macOS — quick commands

```
chmod +x run_tests.sh

# Local Docker (default)
./run_tests.sh

# Local Docker (explicit)
./run_tests.sh --local

# Vite dev server
./run_tests.sh --dev

# Production
./run_tests.sh --prod

# Extra pytest args still work
./run_tests.sh --prod -m smoke
./run_tests.sh --prod
test_auth.py::TestAuthentication::test_TC_AUTH_001_valid_registration -v
```

Manual pytest invocation

```
# Named environment via --env flag
pytest --env=local
pytest --env=dev
pytest --env=prod

# Explicit URL via --base-url flag
pytest --base-url=https://webshop.xpert-hub.com

# BASE_URL environment variable
BASE_URL=https://webshop.xpert-hub.com pytest

# Run a specific file
pytest test_auth.py -v --env=prod

# Run a specific test
pytest test_auth.py::TestAuthentication::test_TC_AUTH_001_valid_registration -v --env=prod
```

```
# Run only tests tagged with a marker
pytest -m cart --env=prod
pytest -m "bug_pd2 or bug_pd4" --env=prod

# Run with all artifacts (screenshots + video + trace)
pytest --env=prod --screenshot=on --video=on --tracing=on
```

URL resolution order

The target URL is resolved in this priority order:

1. `--base-url` flag: `pytest --base-url=https://webshop.xpert-hub.com`
2. `--env` flag: `pytest --env=prod`
3. `BASE_URL` environment variable: `BASE_URL=https://webshop.xpert-hub.com pytest`
4. Default: `http://localhost`

Environment variables

Variable	Default	Description
<code>BASE_URL</code>	<code>http://localhost</code>	Root URL — overridden by <code>--base-url</code> or <code>--env</code> flags

Output

After a run you will find:

Path	Contents
<code>reports/report.html</code>	Self-contained HTML report (open in browser)
<code>test-results/</code>	Playwright artifacts (screenshots, videos, traces)
<code>test-results/screenshots/</code>	Per-test screenshots taken at key assertion points
<code>test-results/videos/</code>	MP4 videos for failed tests

Test Files

`test_auth.py` — Authentication

Tests covering user registration, login, account lockout, password policy, and cart state on logout.

`test_products.py` — Product Display

Tests covering image paths, price filter inclusivity, alphabetical and numeric sorting, out-of-stock badges, and mobile button visibility.

`test_cart.py` — Shopping Cart

Tests covering add-to-cart flow, quantity limits, tax rate, discount codes, total recalculation, and cart persistence across logout.

`test_checkout.py` — Checkout & Purchase Flow

Tests covering login gate, CVV validation, expiry date validation, order confirmation, and order history isolation.

`test_database.py` — Database & API Integrity

API-level tests covering negative quantity rejection, cross-user order data leakage, and password hashing algorithm documentation.

Test Case Reference Table

TC ID	File	Description	Expected on Buggy App	Bug(s)
TC_AUTH_001	test_auth.py	Valid registration with strong password	PASS	—
TC_AUTH_002	test_auth.py	7-char password rejected at registration	FAIL	Auth2
TC_AUTH_003	test_auth.py	No-uppercase password rejected	FAIL	Auth2
TC_AUTH_004	test_auth.py	No-special-char password rejected	FAIL	Auth2
TC_AUTH_005	test_auth.py	Valid login (user2)	PASS	—
TC_AUTH_006	test_auth.py	Invalid login shows error	PASS	—
TC_AUTH_007	test_auth.py	No lockout after 5 failed attempts (documents bug)	PASS (bug documented)	Auth5
TC_AUTH_008	test_auth.py	user1 password violates policy	FAIL	Auth1
TC_AUTH_009	test_auth.py	Cart cleared after logout	FAIL	Cart5
TC_PROD_001	test_products.py	Electronics images use correct /images/ path	FAIL	PD1
TC_PROD_002	test_products.py	Min price filter is inclusive	PASS	—
TC_PROD_003	test_products.py	Max price filter is inclusive (\$29.99 product appears)	FAIL	PD2
TC_PROD_004	test_products.py	Sort by name A-Z is correct	PASS	—
TC_PROD_005	test_products.py	Sort by name Z-A is correct	PASS	—

TC ID	File	Description	Expected on Buggy App	Bug(s)
TC_PROD_006	test_products.py	Sort by price asc is numeric (cheapest first)	FAIL	PD5
TC_PROD_007	test_products.py	Sort by price desc is numeric (most expensive first)	FAIL	PD5
TC_PROD_008	test_products.py	Out-of-stock product shows "Out of Stock" badge	FAIL	PD4
TC_PROD_009	test_products.py	Add to Cart button visible on mobile for Books	FAIL	PD3
TC_CART_001	test_cart.py	Product added to cart appears on /cart	PASS	—
TC_CART_002	test_cart.py	Max 10 items limit per product enforced	FAIL	Cart1
TC_CART_003	test_cart.py	VAT is 10% of subtotal	FAIL	Cart2
TC_CART_004	test_cart.py	TEST10 discount code is exactly 10%	FAIL	Cart3
TC_CART_005	test_cart.py	STUDENT20 discount code is exactly 20%	PASS	—
TC_CART_006	test_cart.py	Cart total updates after item removal	FAIL	Cart4
TC_CART_007	test_cart.py	Cart empty after logout	FAIL	Cart5
TC_CHECKOUT_001	test_checkout.py	Checkout requires login	PASS	—
TC_CHECKOUT_002	test_checkout.py	5-digit CVV rejected	FAIL	Purchase2
TC_CHECKOUT_003	test_checkout.py	Expired card date rejected	FAIL	Purchase3
TC_CHECKOUT_004	test_checkout.py	Order confirmation message shown	FAIL	Purchase5
TC_CHECKOUT_005	test_checkout.py	Order history shows only own orders	FAIL	Purchase4
TC_DB_001	test_database.py	Negative cart quantity rejected (API)	FAIL	DB3
TC_DB_002	test_database.py	Orders API returns only own orders	FAIL	Purchase4 / DB1
TC_DB_003	test_database.py	Password not stored as MD5 (xfail — documents bug)	XFAIL	Auth3

Total: 33 test cases

- Expected PASS (no bug): 7
- Expected FAIL (bug present): 25
- Expected XFAIL (documented, cannot auto-verify): 1

Markers

Run `pytest --markers` to see the full list. Key markers:

Marker	Purpose
<code>auth</code>	Authentication tests
<code>products</code>	Product display tests
<code>cart</code>	Cart tests
<code>checkout</code>	Checkout tests
<code>database</code>	DB / API tests
<code>smoke</code>	Critical path (quick sanity check)
<code>bug_auth1 ... bug_db3</code>	Tag tests to specific known bugs

Example — run only smoke tests:

```
pytest -m smoke -v
```

Example — run all cart-related bug tests:

```
pytest -m "bug_cart1 or bug_cart2 or bug_cart3 or bug_cart4 or bug_cart5" -v
```

Credentials

User	Password	Notes
user2	Password@2	Primary test user — policy-compliant password
user1	password123	Weak password — violates policy (BUG Auth1)

Architecture Notes

- **conftest.py** provides all shared fixtures and helper functions.
- Each test file is a single class inheriting no base class (plain pytest style).
- Screenshots are taken at key assertion points within every test, not only on failure.
- The `pytest_runtest_makereport` hook in `conftest.py` captures an additional failure screenshot automatically.

- Cart state is cleared before each cart/checkout test using `page.evaluate("localStorage.clear()")` and the **DELETE** `/api/cart/clear` API endpoint.